UART

PC-FPGA communication

Asynchronous Serial Communication

In this course we will study the "Asynchronous Serial Communication", one of the most common protocol for slow low-level data transfer.

One of the most known standard using this type of communication is the RS-232.



Asynchronous Serial Communication

As the name suggests, this type of communication:

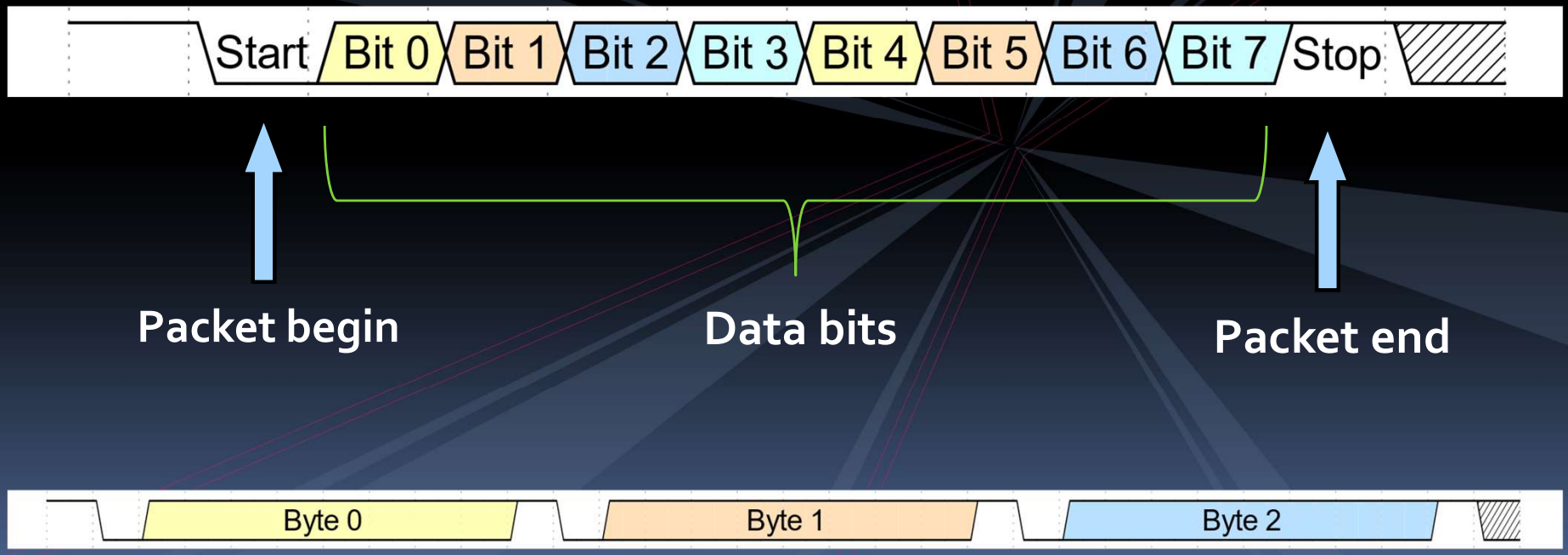
- has no dedicated clock line,
- Transmits the bits one-by-one.

We will use the full-duplex point-to-point version, so with just two devices.



Asynchronous Serial Communication

Data is encapsulated in "packets" of (typically) 8 bits each, with a "Start" bit at the beginning and a "Stop" bit at the end.



Asynchronous Serial Communication

There exist many different versions of this kind of communication, with different:

- baud rate (also called "symbol rate"), which is the "total" number of bits (including start and stop) per second (1200, 9600, 57600, 115200, 230400, 2000000, …)
- number of data bits (5, 6, 7, 8 or 9)
- parity (no parity, even or odd)
- number of stop bits (1, 1 and half, 2)

Asynchronous Serial Communication

For now, we will just use the most common version of the asynchronous serial communication:

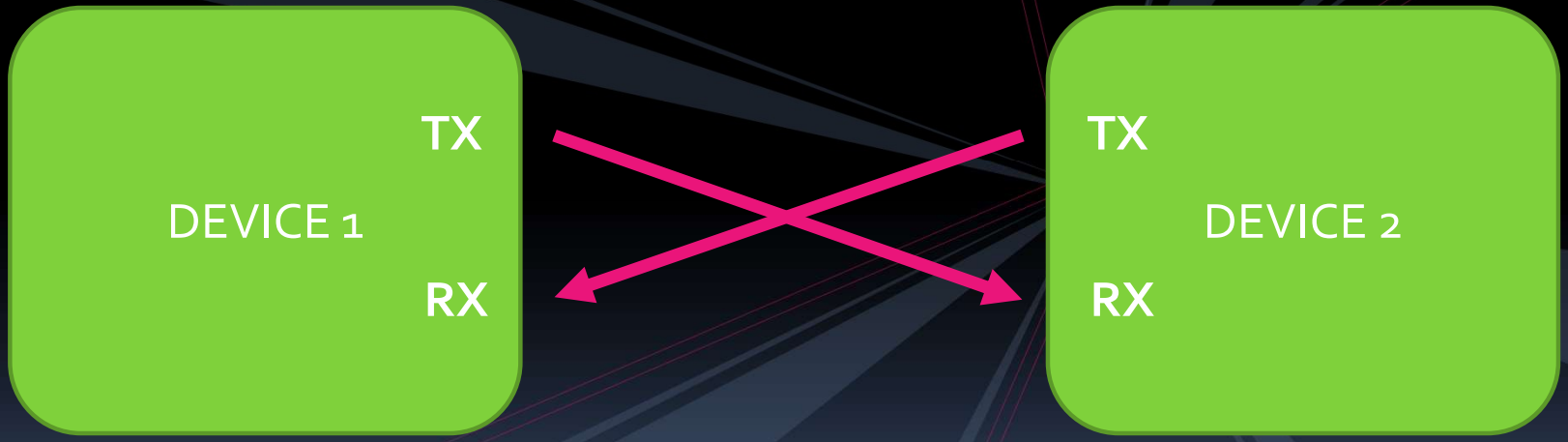
- 115200 bits per second
- 8 data bits per packet
- no parity
- 1 stop bit

often called:

115200 8N1

Signals

The two most important signals are the TX and RX signals (sometimes called TXD and RXD), which transmit the data.



The TX of one device must connected with the RX of the other one!
This is a very common source of errors

Auxiliary signals

There are also many "auxiliary" signals for:

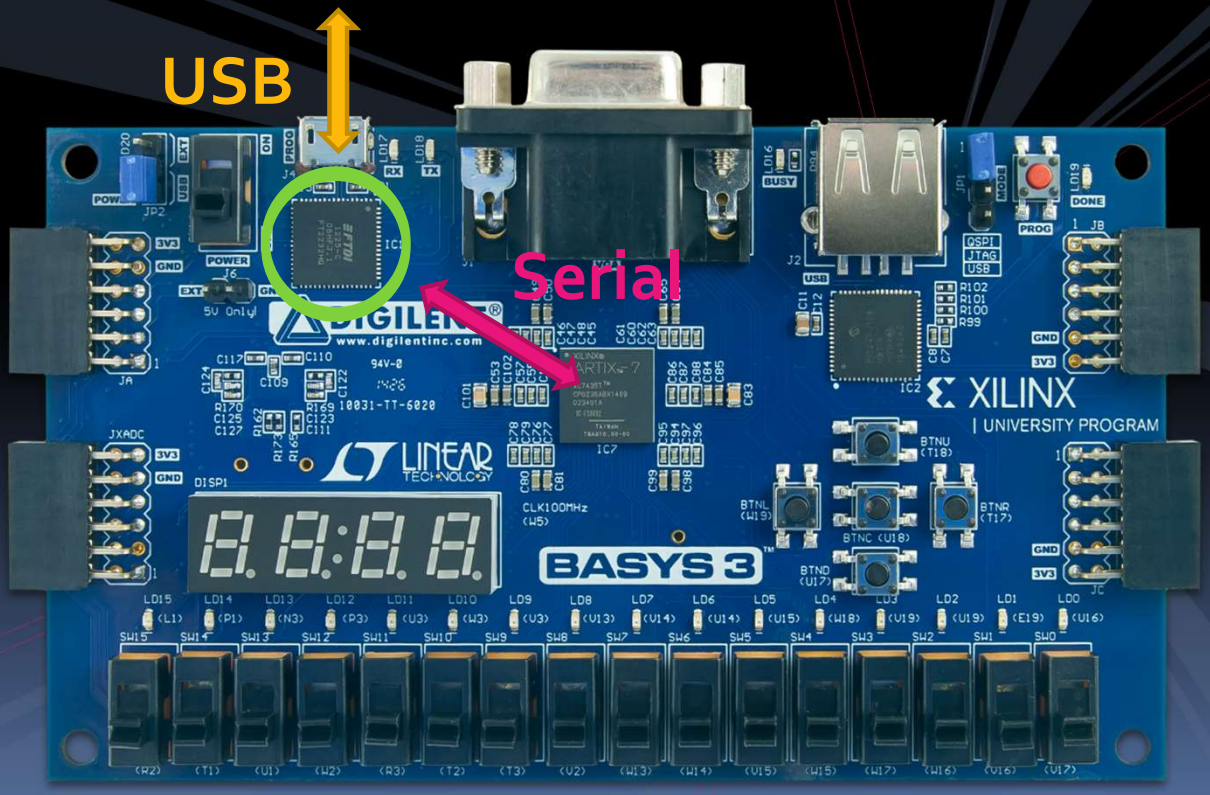
- hardware flow control (RTS, CTS, DSR, DTR)
- presence detection (DCD)
- bell ringing (RI)
- …

which we will not use.

We will use just the plain 115200 8N1 communication without flow control.

Serial communication on Basys 3

On the Digilent Basys 3 board there is a USB to serial converter (FTDI FT2232H):



Serial communication on Basys 3

The TX and RX pins of the converter are connected to pins **B18** and **A18** of the Basys 3 FPGA.

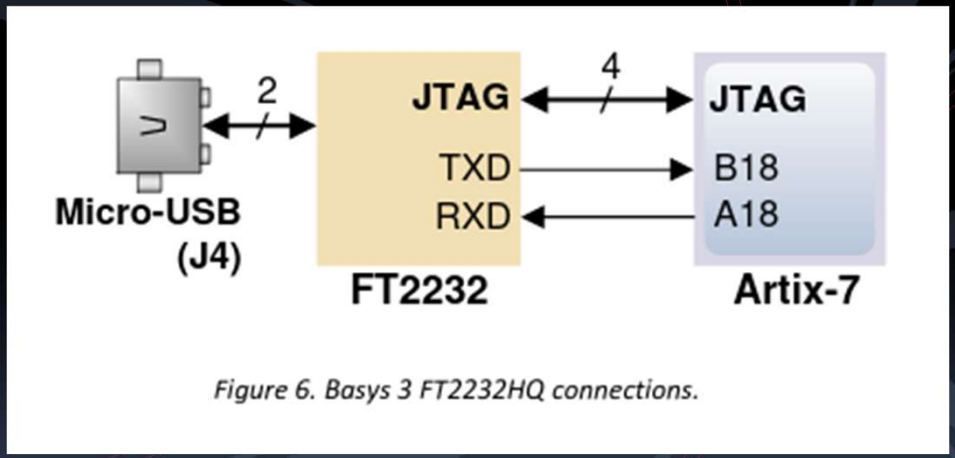


Figure 6. Basys 3 FT2232HQ connections.

Basys 3: serial LEDs (1/2)



These two LEDs turn on when data is transmitted (TX) or received (RX), from the PC "point of view".

They are driven directly by the USB-serial converter so, even if your firmware is broken (or missing), they still work.

Basys 3: serial LEDs (2/2)

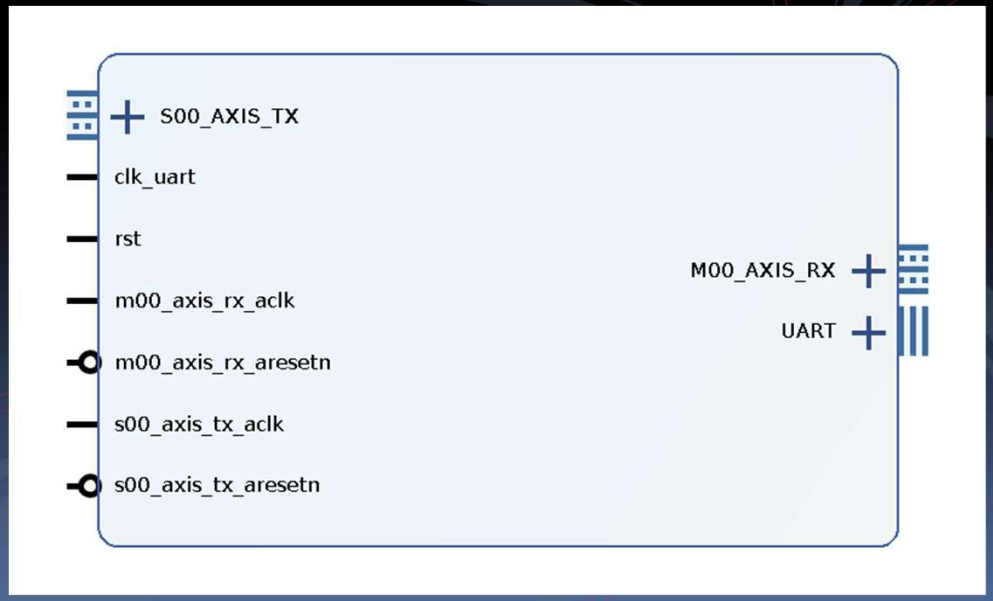


They are very useful to debug simple issues, like:

"Did I select the correct serial port in the serial terminal?"

Asynchronous serial IP-Core

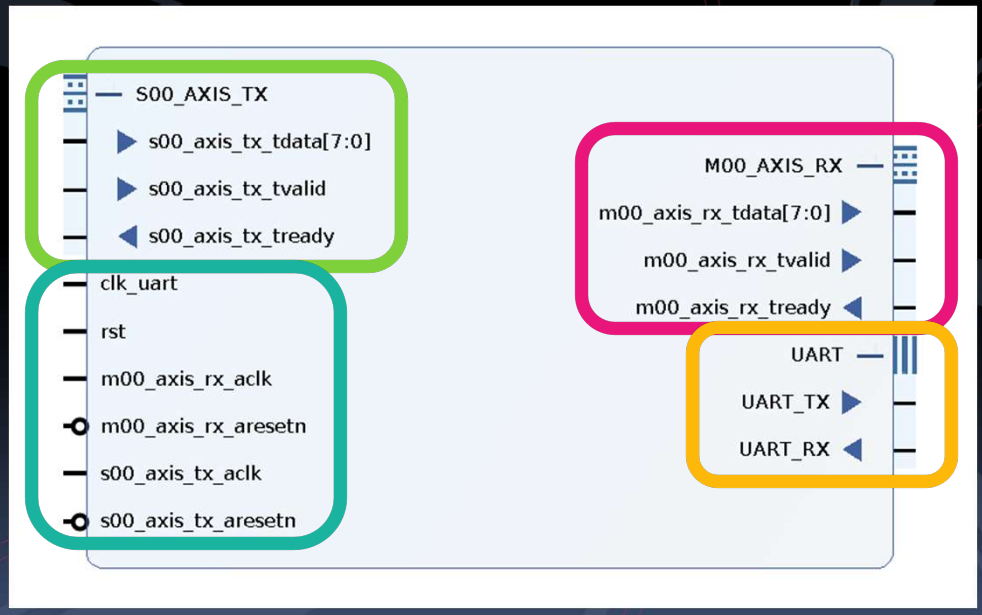
To “talk” the asynchronous serial protocol we will use a UART (Universal Asynchronous Receiver-Transmitter) IP-Core with an AXI4-Stream interface:



Asynchronous serial IP-Core

Slave AXI4-Stream port:
data coming into this port is converted and then transmitted through TX

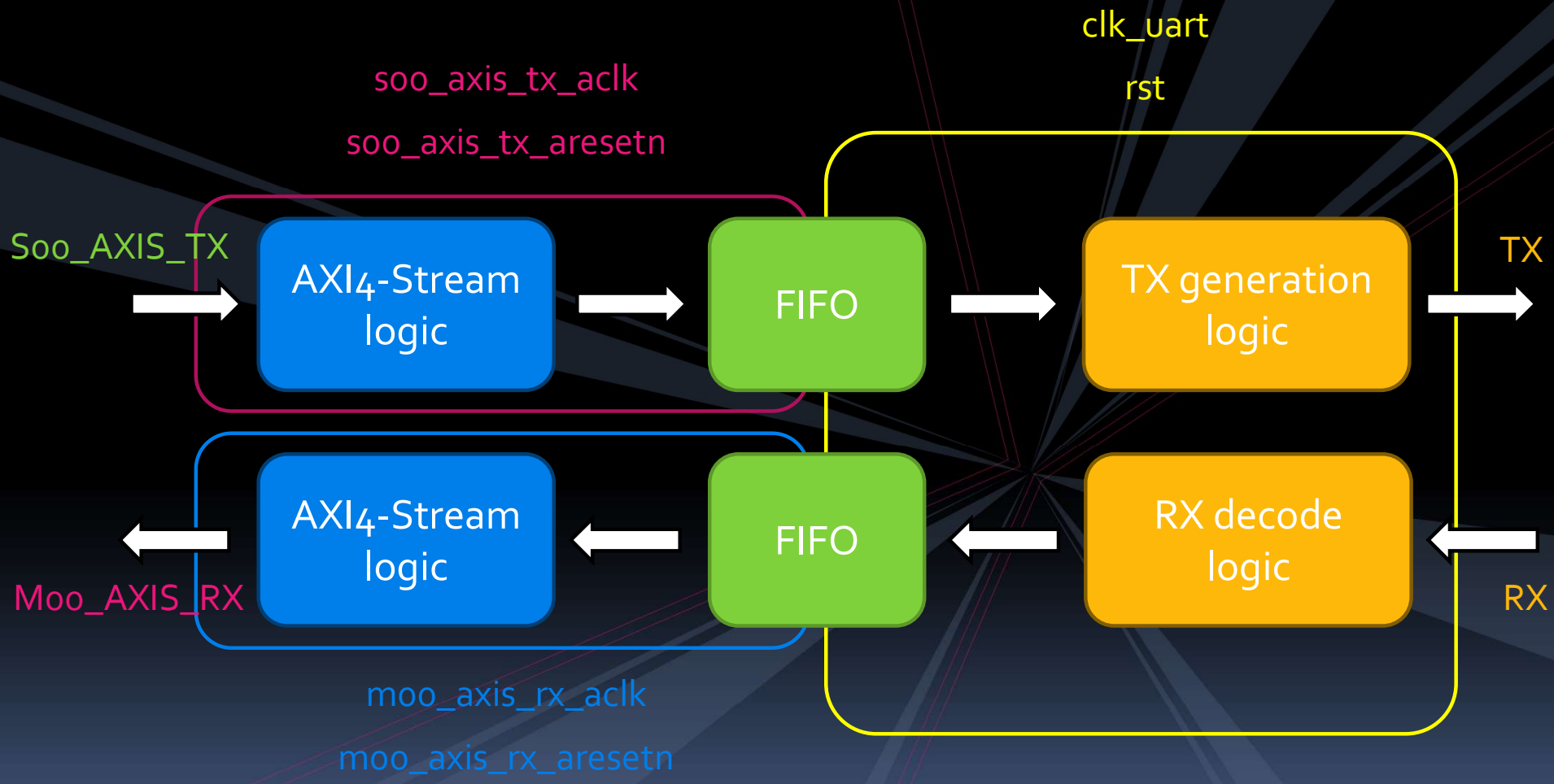
Master AXI4-Stream port:
data received from the RX signal is converted and then transmitted from this port



Clocks and reset signals

Asynchronous serial signals, to be connected to the FPGA I/O pins

Asynchronous serial IP-Core



Serial devices on the PC

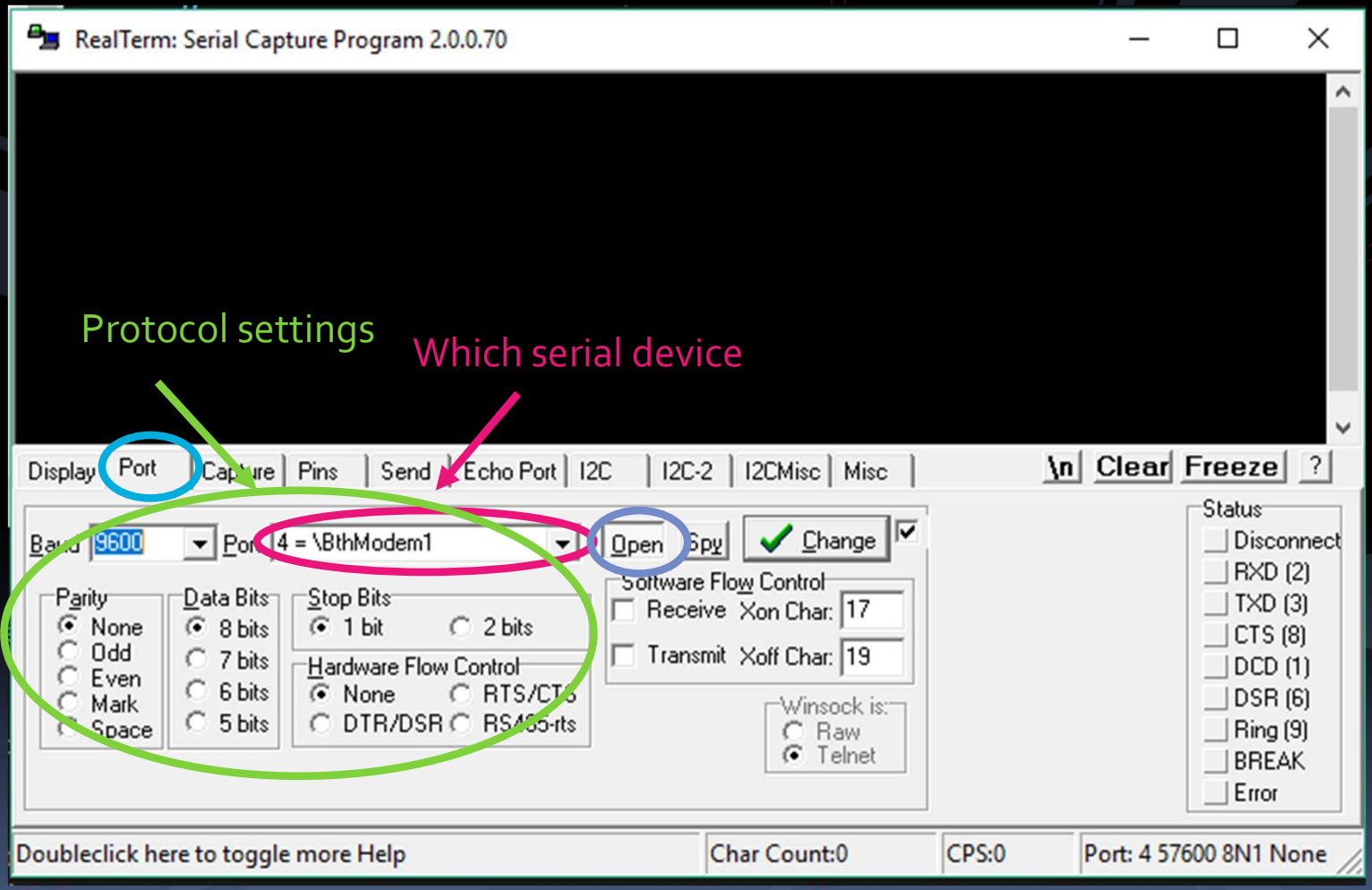
On the PC, USB-serial converters are seen as:

- Windows: COMx ports
- Linux: /dev/ttyUSBx devices
- Mac OS: /dev/tty.usbserial-x devices

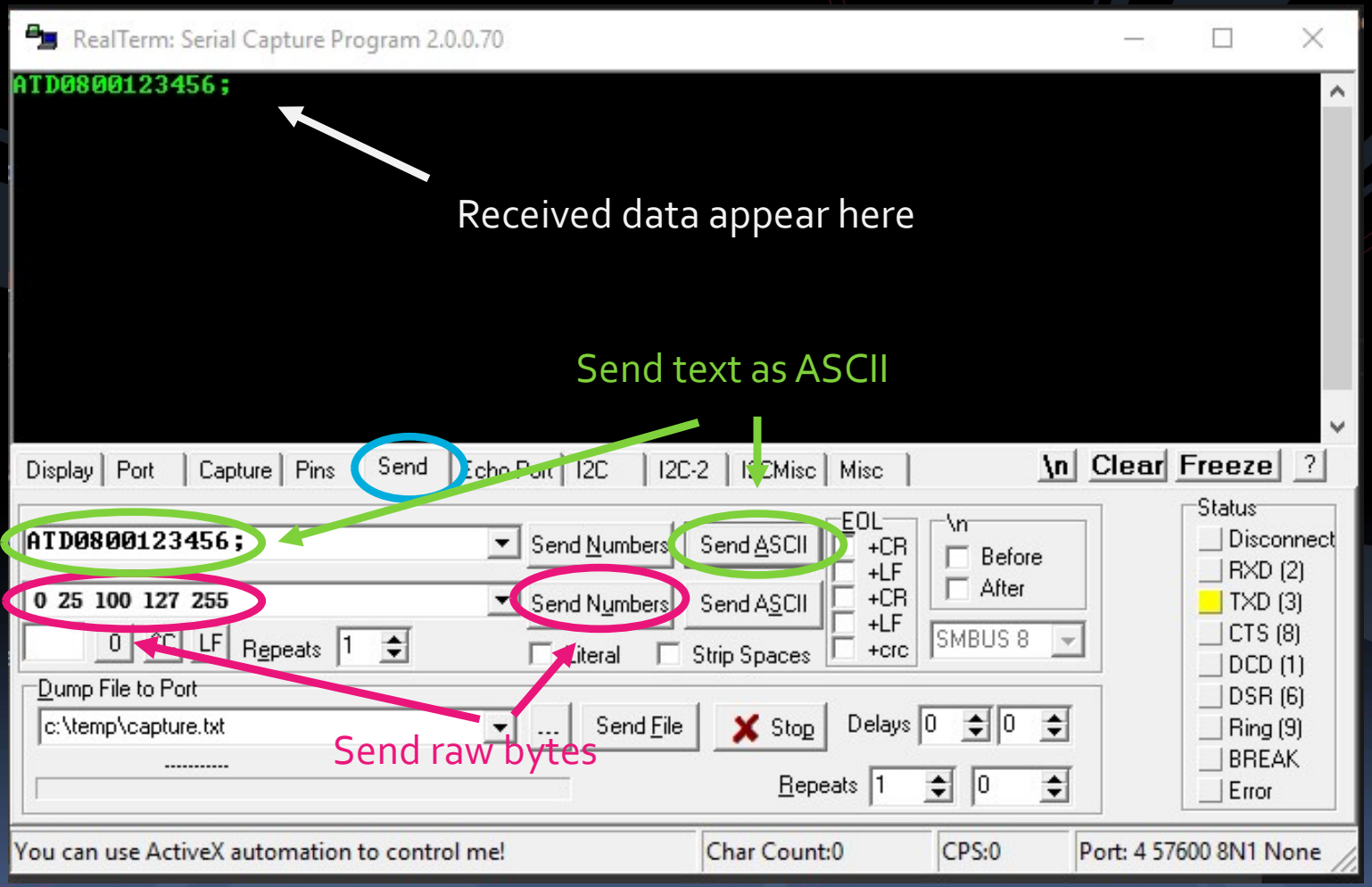
To interact with a serial port, you need to write your own software or use a Serial terminal like:

- RealTerm (Windows)
- Cutecom (Linux)

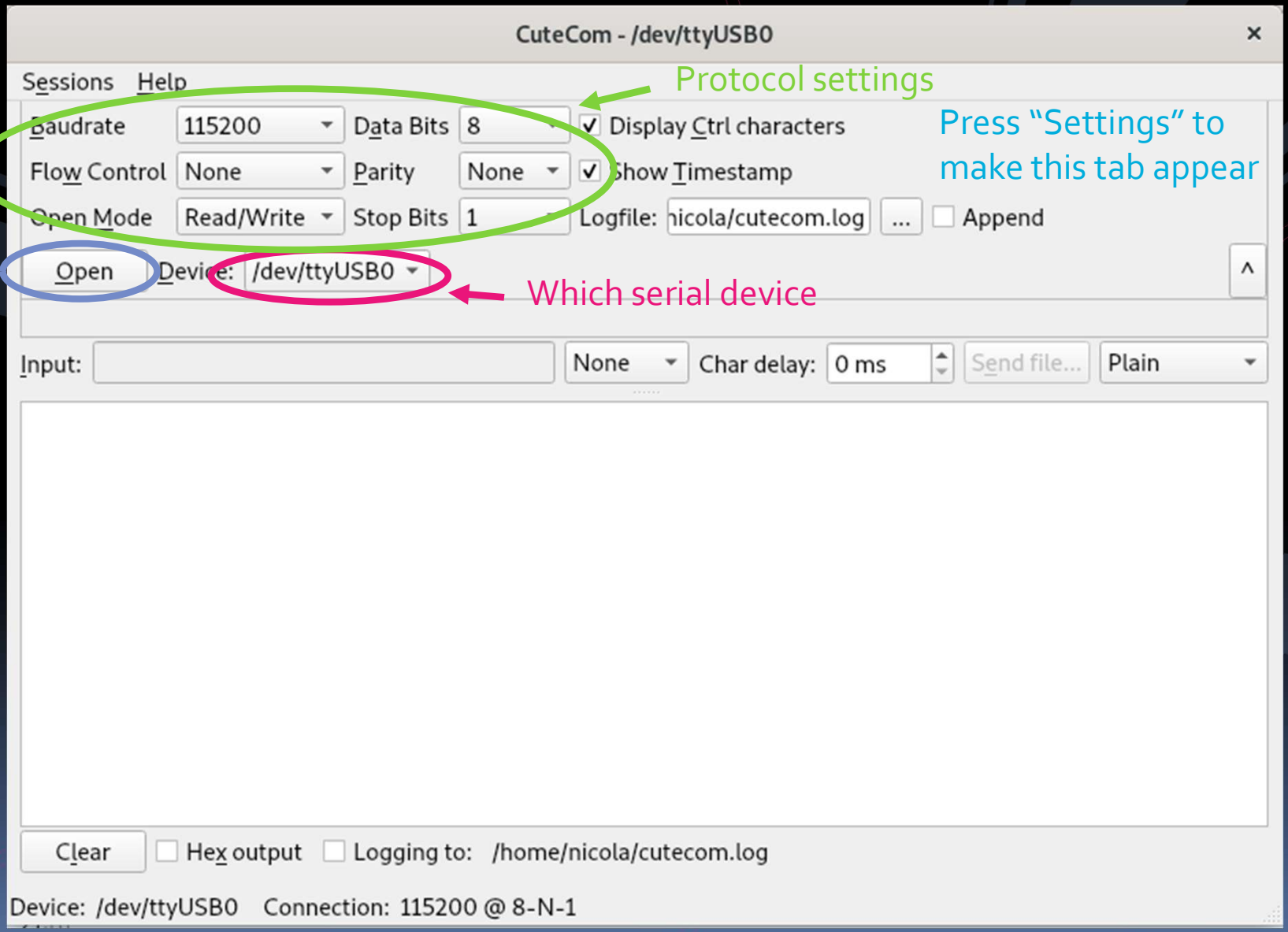
RealTerm – open port



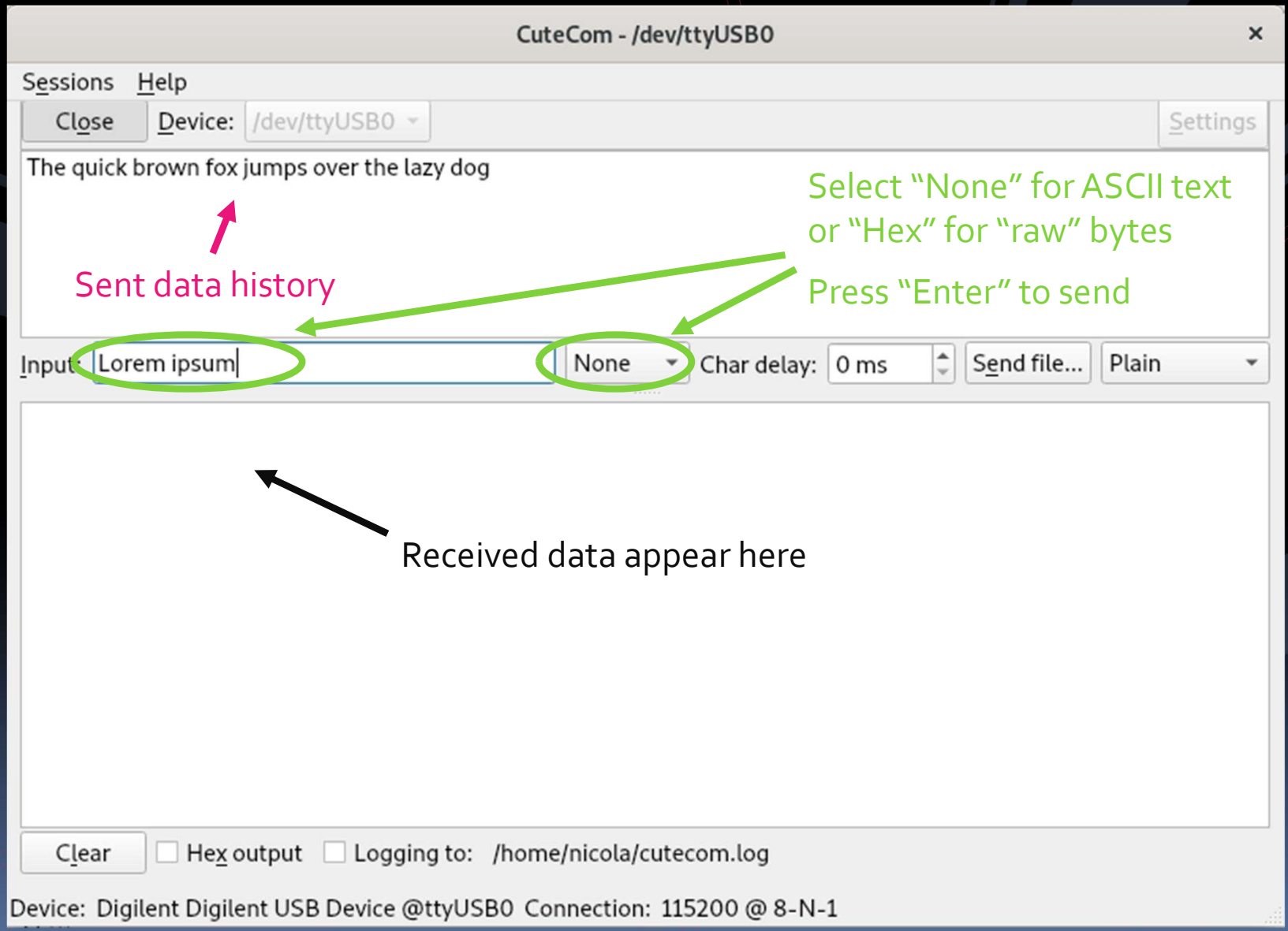
RealTerm – send and receive data



Cutecom – open port



Cutecom – send and receive data



UART

EXERCISE

Serial communication

Using the AXI4-Stream UART module, build a LED-Switch serial interface:

- When a byte is transmitted from the PC to the FPGA, turn on the corresponding LEDS (LD[0:7])
- When a Push button is pressed on the board (BTND), send to the PC the status of the switches (SW[0:7])

Serial communication hints

- First, start by testing the IP-Core in a loopback configuration (input interface connected to the output interface) with a serial monitor
- You have to send 1 «switch status byte» for every button press: it must edge-sensitive, not level-sensitive (without using «if rising_edge(button)» of course)
- You may have some issues on the push button due to the bouncing: try to solve them

Serial communication scheme

